GEORG-AUGUST-UNIVERSITÄT GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

HPS

Seminar Report

A performance comparison of a machine learning optimizer in CUDA and OpenCL

Lennart Hahner

MatrNr: 16853416

Supervisor: Zoya Maish

Georg-August-Universität Göttingen
Institute of Computer Science

March 9, 2026

Abstract

Graphics Processing Units (GPUs) play a central role in accelerating machine learning workloads due to their ability to efficiently execute data-parallel computations. Several programming frameworks exist to exploit GPU hardware, with CUDA and OpenCL being two widely used solutions. While CUDA is optimized for NVIDIA hardware, OpenCL provides a vendor-neutral and cross-platform programming model. This raises the question of how both frameworks compare in terms of performance for machine learning workloads.

This report presents a performance comparison of CUDA and OpenCL using an implementation of the Adam optimization algorithm applied to a matrix multiplication workload (GEMM) that simulates neural network training operations. A modular benchmark framework was implemented in C++ with a layered architecture to execute the optimizer in both frameworks under identical conditions.

The evaluation measures GPU computation time and host–device data transfer time across three NVIDIA GPUs representing different hardware generations: the GTX 970, Quadro RTX 5000, and A100-SXM4. The results show that OpenCL achieves slightly faster computation times in most experiments, while CUDA often performs better in data transfer operations. Overall, both frameworks demonstrate comparable performance, highlighting the trade-off between hardware-specific optimization and cross-platform portability.

Declaration on the use of ChatGPT and comparable tools in the context of examinations

In this work I have used ChatGPT or another AI as follows:

- ☐ Not at all
- ☐ During brainstorming
- ☐ When creating the outline
- ☑ To write individual passages, altogether to the extent of 0% of the entire text
- ☑ For the development of software source texts
- ☐ For optimizing or restructuring software source texts
- ☐ For proofreading or optimizing
- ☐ Further, namely: -

I hereby declare that I have stated all uses completely.
Missing or incorrect information will be considered as an attempt to cheat.

Contents

List of Tables

List of Figures

List of Listings

List of Abbreviations

HPC High-Performance Computing

SIMD Single Instruction Multiple Data

GPU General Graphic Processing Unit

1 Introduction

Training neural networks typically requires large-scale datasets such as ImageNet or MNIST to achieve robust generalization. Because training involves multiple passes over these datasets and computationally expensive gradient-based optimization, the process is highly data-intensive.

To process such workloads efficiently, data parallelism is required. Data parallelism refers to applying the same arithmetic operations to multiple data elements simultaneously. Although modern CPUs support limited forms of data parallelism, their architectures are primarily optimized for low-latency and control-intensive tasks rather than high-throughput parallel computation [**kirk2016parallel**]. General Graphic Processing Unit (GPU)s, in contrast, follow a Single Instruction Multiple Data (SIMD) execution model that enables a much higher degree of data parallelism. Consequently, GPUs have become essential for efficient neural network training. Several hardware vendors provide different software frameworks to exploit GPU capabilities.

This work compares GPU computing frameworks such as OpenCL and CUDA, which allow developers to utilize the computational capabilities of GPUs. More specifically, the frameworks are compared using a benchmark program. To facilitate this comparison, the Adam optimization algorithm [**kingma2015adam**] is implemented in each framework. This implementation enables an evaluation of the performance characteristics of the different approaches.

CUDA is a widely used framework for GPU computing but is restricted to NVIDIA hardware. In addition, CUDA is neither standardized nor fully cross-platform. Nevertheless, it remains the dominant solution for neural network training, as it is integrated into frameworks such as PyTorch, Keras, and TensorFlow. This creates a strong dependency on NVIDIA GPUs, since hardware from other vendors, such as AMD or ARM, is not compatible with the CUDA API [**nvidia2024cuda**].

In contrast, OpenCL provides a standardized, cross-platform parallel computing API based on C and C++. It is an open standard maintained by the Khronos Group. This report therefore evaluates the performance of a vendor-neutral solution in comparison with an NVIDIA-specific framework. However, the experiments are executed on an NVIDIA GPU.

The Adam optimizer is applied to a GEMM workload that simulates the training process of a neural network. The model architecture, training procedure, and hyperparameters remain identical across all experiments; only the optimizer implementation differs between the CUDA- and OpenCL-based approaches.

The remainder of this paper is structured as follows. Section **??** provides an overview of related work. Section **??** describes the implementation of the benchmark program. Finally, Section **??** presents the benchmark setup and discusses the results.

2 Related Work

Several prior studies have compared OpenCL and CUDA. This section provides an overview of the considered comparison aspects, the evaluated workloads, and the main findings reported.

Du P. et al. [**Du2011OpenCLCUDA**], compars CUDA and OpenCL with respect to syntax, cross-platform compatibility, and overall computation and data transfer time. The evaluation is based on triangular solver (TRSM) and matrix multiplication (GEMM) workloads, with a particular focus on OpenCL performance across different platforms, including NVIDIA and ATI devices. The authors highlight OpenCL's functional portability, but demonstrate that performance portability is often limited by low-level architectural details. They attribute these limitations to differences in memory hierarchies, compiler optimizations, and architectural execution models.

Furthermore, Fang J. et al. [**Fang2011CUDAOpenCL**], are evaluating performance is evaluated using a tailored performance ratio, defined as the ratio between the performance achieved by OpenCL and that achieved by CUDA. The study measures both device memory bandwidth and floating-point performance. In addition, similar to [**Du2011OpenCLCUDA**], the authors provide a conceptual comparison of the two frameworks, focusing on differences in terminology and programming abstractions used by CUDA and OpenCL. Performance is evaluated on 16 real-world workloads, including graph traversal, matrix transposition, and sparse matrix–vector multiplication. The experiments are conducted on multiple platforms from different vendors, such as NVIDIA and AMD. The results show that OpenCL's portability can lead to performance degradation in certain scenarios. One identified reason is that, on CPUs, OpenCL memory objects are implicitly cached by the hardware, making explicit use of local memory unnecessary and potentially harmful due to the introduced overhead.

In another comparison by Karimi K. et al. [**https://arxiv.org/abs/1005.2581**], OpenCL and CUDA are compared with respect to data transfer times to and from the GPU, kernel execution times, and end-to-end application execution times for both CUDA and OpenCL. Similar to other studies, they also compare CUDA and OpenCL conceptually in terms of the code changes required to port an application from CUDA to OpenCL. They report that the porting effort required only minor changes. As a workload, they consider an adiabatic quantum algorithm, which is a Monte Carlo simulation of a quantum spin system written in C++. For their benchmark, they conclude that choosing CUDA may be the better option when performance is of primary importance. Furthermore, they state that the choice between CUDA and OpenCL depends on the available hardware on the client side and the supporting development tools.

Rather than presenting a comparison, Fang J. et al. [<**empty citation**>] describe an implementation of a neural network using OpenCL. They measure the total amount of local memory per work-group and the speed-up achieved compared to sequential training on a CPU, with respect to the number of neurons, the number of samples, and the number of layers. Of particular interest is their experimental setup, in which they train a multilayer perceptron using a particle swarm optimizer (PSO). They emphasize the portability of OpenCL and conclude that training with parallel backpropagation on a GPU is only recommended for smaller networks.

Most of the presented work was published within the time frame from 2010 to 2012. Furthermore, the workloads considered are mostly non–machine-learning-related. Therefore,

another motivation for this paper is to revisit these frameworks and examine what may have changed in both frameworks over the past 13 years. Additionally, as stated in Section **??**, a neural network workload provides a contemporary and widely used example of GPU applications.

[**https://www.sciencedirect.com/science/article/abs/pii/S0167819111001335**], x [**https://ieeexplore.ieee.org/document/6047190**], x [**https://arxiv.org/abs/1005.2581**]

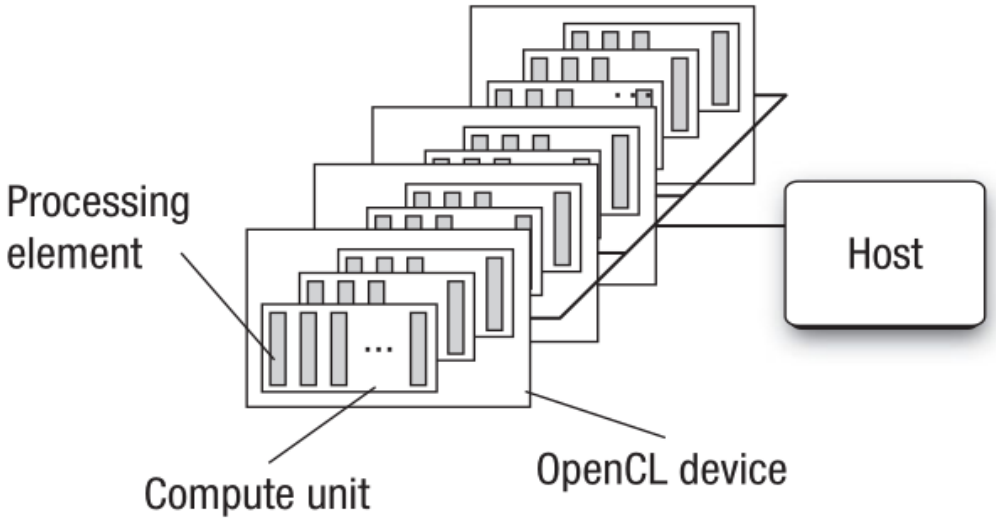

Figure 1: The interaction with the host and OpenCL devices which are possible GPUs is displayed here. In a GPU and CPU setup the Host is considered to be the CPU. [**opencl-programming-guide**]

3 Conecptual differences of OpenCL and CUDA

To introduce the GPU interface frameworks OpenCL and CUDA this chapter should provide a conceptual comparison of the software implementation and especially highlight the differences in terms of usability.

Both frameworks present different abstractions on how the interaction with the computing device (GPU) is structured. In OpenCL we differ between the host and kernels. While a host is the application which calls the kernels and is considered to be the execution unit which also compiles and runs the main program. On the other hand the kernel is considered as the part which is executed with the OpenCL runtime on a computing device or computing unit. Each running instance of a kernel is identified as a work-item.

While running the OpenCL runtime creates an index-space which associates each work-item with its corresponding coordinates inside the index-space. These work-items are grouped in work groups. The index space spans an N-dimensioned range of values and is called an NDRange. Inside an OpenCL program, an NDRange is defined by an integer array of length N specifying the size of the index space in each dimension. The figure 4 demonstrates the structes of the index space in OpenCL

CUDA has a similar model as OpenCL. Consisting of a kernel and a host. The host is considered as the computing unit which compiles the code and runs it without or little data parallelism. The kernel code runs code mostly in data parallelism using the ANSI C programming language with CUDA extention. During execution the kernel code is moved to a device which is for example a GPU. The kernel spawns threads needed for execution those are chunked into grids.

These grids are similar as the OpenCL NDRange. All threads in a grid execute the

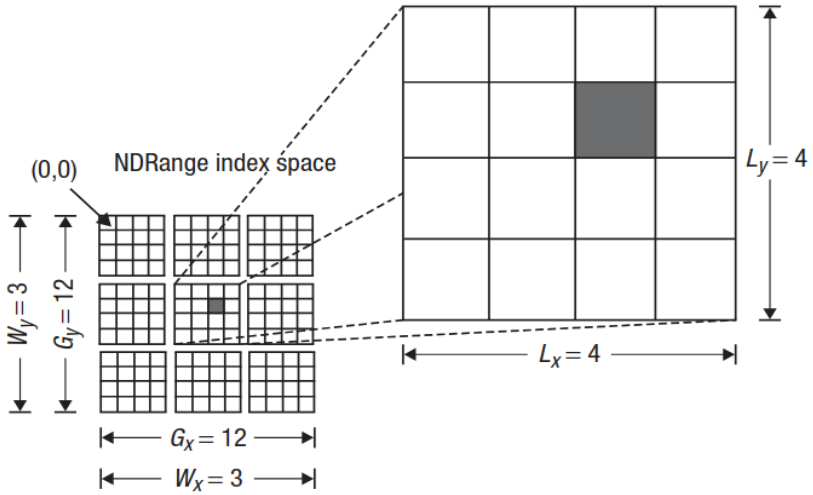

Figure 2: The work-item resides in an $N \times N$ index-space. Where the shades box is one workitem at location $(6, 5)$ inside the work-group $(1, 1)$. Overall the size of the NDRange index space is 12 divided in 3 work-groups. While each work-group has 4 work-items. [**opencl-programming-guide**]

same kernel function. The threads rely on unique coordinates, similar as the index in the NDRange. The coordinates consiting of a block index and a thread index. Figure **??** shows a simple example of the CUDA thread organization. The first grid consists of four blocks while one block consists of sixteen threads. Each grid has a total of $N * M$ threads. In general a grid is organized as a 2D array of blocks which are organized into a 3D array of threads. [**programming-massively-parallel-processors**]

Required for a host to interact with a device in OpenCL is its context. The programmer is required to define a context about the environment within the host is running in, like available devices, kernels to run, program objects and memory objects. The developer can issues commands to the command-queue, these are either kernel execution commands, memory commands or synchronization commands.

Different to the OpenCL context CUDA handles most of the context for a specific device by the initalized runtime. Mainly using the function `cudaInitDevice()` the runtime is created and also the context for the device on which the user executes the kernel. [**cuda-programming-guide**]

OpenCL memory is allocated within a specific context, which also defines the set of devices that can access this memory. There are two major types of memory objects that can be allocated in OpenCL: buffers and images. Memory objects created within a context are visible to all devices associated with that context, enabling shared data access across devices.

To create a buffer, the function `clCreateBuffer` is used. Once created, buffer objects are passed as arguments when creating kernel objects, allowing kernels to read from and write to the buffer during execution. In addition, OpenCL supports subdividing a buffer into smaller regions called sub-buffers. This makes it possible to partition the data so that each device can operate on a separate sub-buffer, which can improve parallelism and memory management.

Reading from and writing to buffers is performed through a command queue. For this

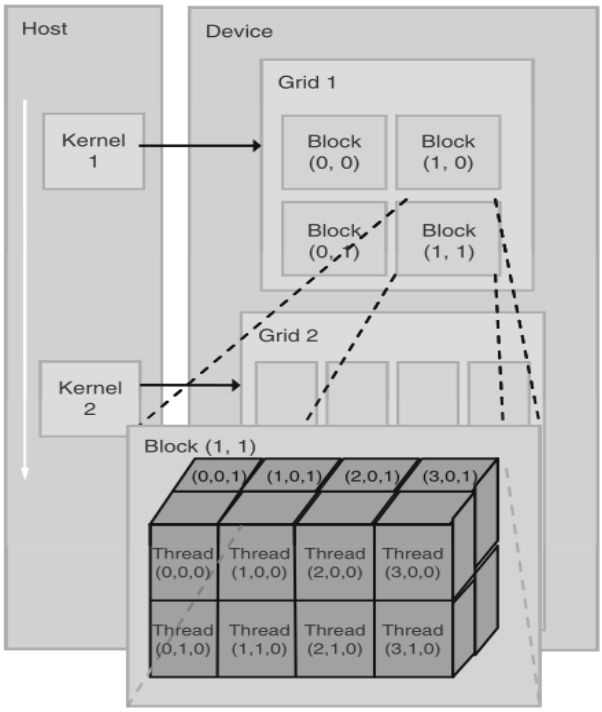

Figure 3: A multidimensional illustration of CUDA grid organization [**opencl-programming-guide**]

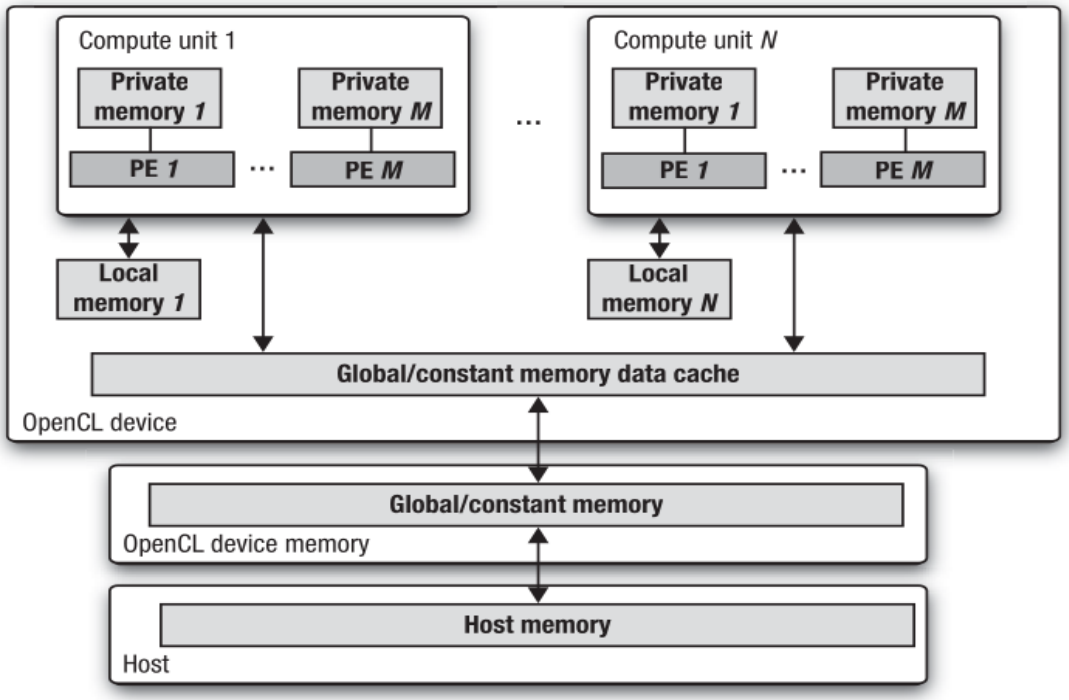

Figure 4: The given image presents all of the architecture parts of the OpenCL framework with considration of the memory model presented. [**opencl-programming-guide**]

purpose, the functions `clEnqueueWriteBuffer` and `clEnqueueReadBuffer` are used to transfer data between host memory and device memory in a controlled and asynchronous manner.

Image memory objects in OpenCL are primarily intended for storing structured data such as image dimensions, pixel layout, and image format information. They are optimized for spatial access patterns and are commonly used in image and signal processing applications. OpenCL supports both 2D and 3D image objects, making them suitable for a wide range of image-processing and volumetric data workloads. [**openCL-programming-guide**]

Same as in OpenCL CUDA seperates the devices memory from the one of the host and the one of the device. Further the devices and the host transfer the data via global memory of the device. The programmer needs to allocate memory on the device in the same as done on the host using the C CUDA extention `cudaMalloc`. But instead of issuing a command queue for copying the data to the host the programmer copies the data with `cudaMemcpy`.

The most important difference between CUDA and OpenCL are portability possibilities. While CUDA relies on Nvidia graphic cards and chips, OpenCL has cross-platform protability. [**programming-massively-parallel-processors**]

Due to CUDAs dependency on NVIDIA GPUs the framework is also well optimized on NVIDIA GPUs. Mainly due to PTX which is NVIDIA device specific parallel thread execution virtual machine instruction set architecture to expose a NVIDIA GPU as a data-parallel executing device. [**https://docs.nvidia.com/cuda/parallel-thread-execution/**] Further it has the benefit of cuDNN which enables hardware tailored operations for common machine learning arithmetics, like convolutions and scheduling certain core architectures like tensors for specific tasks during training. [<**empty citation**>]

The key difference here is that OpenCL doesn't have that device dependency and thus does not bring this specific optimization for a hardware architecture from any vendor. This is broad by abstraction due to key elements like OpenCL platforms, devices and memory models. Since the programmer needs to query and allocate platforms by themself it enables a cross-platform protability of kernels. Which also includes NVIDIA devices. [**https://www.khronos.org/opencl/?utm_source=chatgpt.com**] But OpenCL's hardware abstraction forces vendors to implement flexible compilers/runtimes that can map a single kernel representation onto diverse execution and memory hardware structures resulting in less tailored hardware optimization like in CUDA. [**https://registry.khronos.org/OpenCL**

4 Benchmark implementation

4.1 Architecture

To benchmark and compare OpenCL to CUDA a benchmark program was implemented in C++ and the build-tool CMake. Which follows a layered architecture. [**software-architecture-patter**
Each layer represents a subsystem responsible for a specific set of tasks. This design enables flexible combinations of workloads and optimizers and allows the system to be extended beyond machine learning workloads to support other computational tasks for performance evaluation. Figure 5 provides an overview of the architecture and its modules.

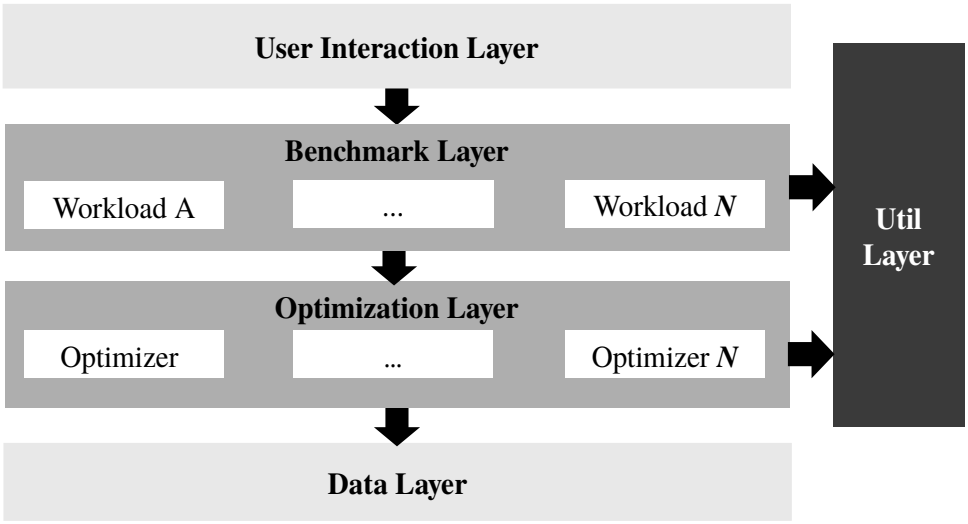

Figure 5: Layered architecture of the benchmark system. The User Interaction Layer (CLI) communicates with the Benchmark Layer, which manages workloads. The Optimization Layer executes workloads using different optimizers, while the Util Layer provides shared functionality.

Program execution starts through the command-line interface (CLI), which initiates the benchmark run. The CLI triggers the Benchmark Layer, which creates the benchmark environment and instantiates the predefined workload. Each workload represents a computational task that is executed by an optimizer. During execution, the Optimization Layer applies the corresponding optimizer to the workload and performs the iterative optimization steps. Throughout the execution, shared functionality provided by the Util Layer is used for tasks such as data handling, framework abstraction, and logging of benchmark results. The following subsections describe the individual layers in more detail.

4.1.1 Benchmark Layer

The Benchmark Layer is the entry point of the application and defines and executes workloads. It consists of two modules: the benchmark module and the workload module.

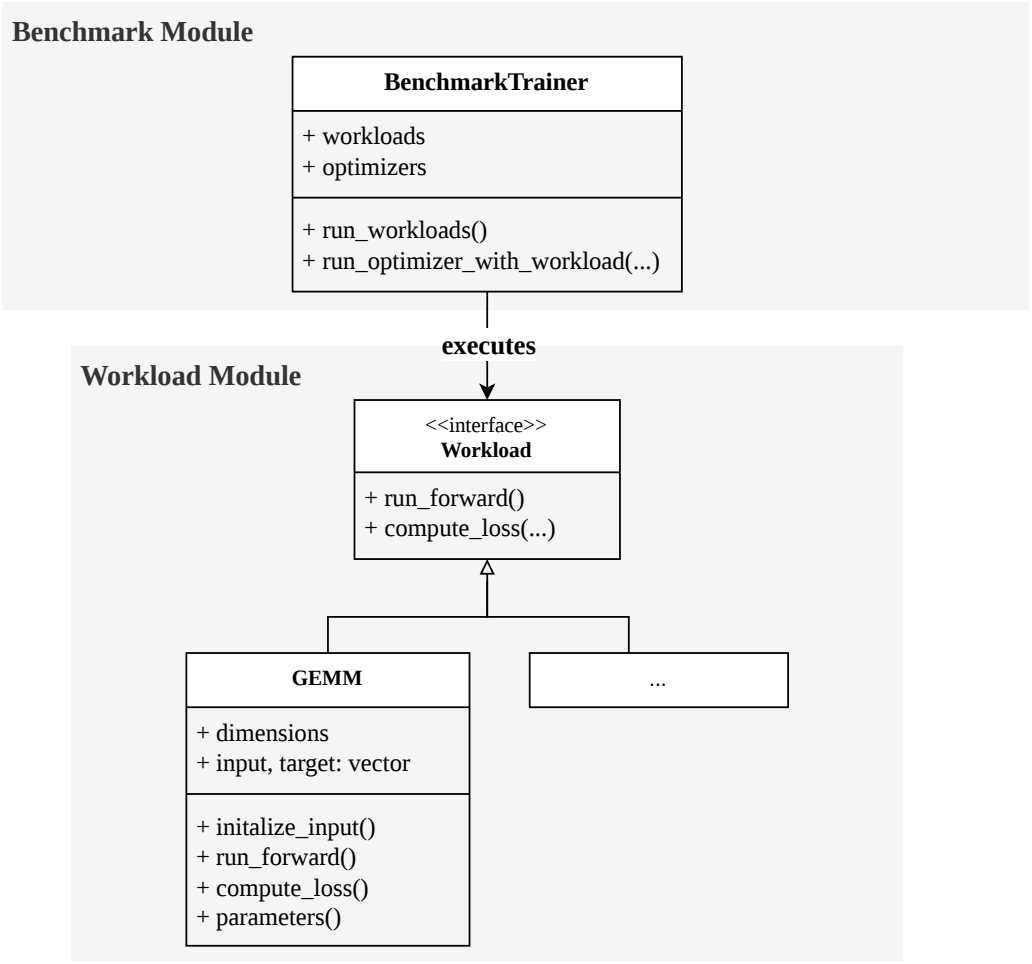

Figure 6: Domain-agnostic class Workload design to enable maintainable integration of additional workloads.

Figure 6 shows the structure of the Benchmark Layer. The class `BenchmarkTrainer` allocates workloads and performs the required preprocessing steps before executing the optimizer. Each workload implements the abstract interface `Workload`. This abstraction decouples workload implementations from the benchmarking and optimization logic, allowing the framework to support different workload types. Within the scope of this project, the class `GEMM` implements a simple neural-network workload consisting only of a forward pass. Since no backpropagation is performed, the workload effectively corresponds to a general matrix multiplication (GEMM) operation. The primary purpose of this workload is to evaluate the performance of large matrix multiplications executed on GPU devices.

4.1.2 Optimization Layer

The Optimization Layer contains the optimizer implementations applied to workloads by the `BenchmarkTrainer`.

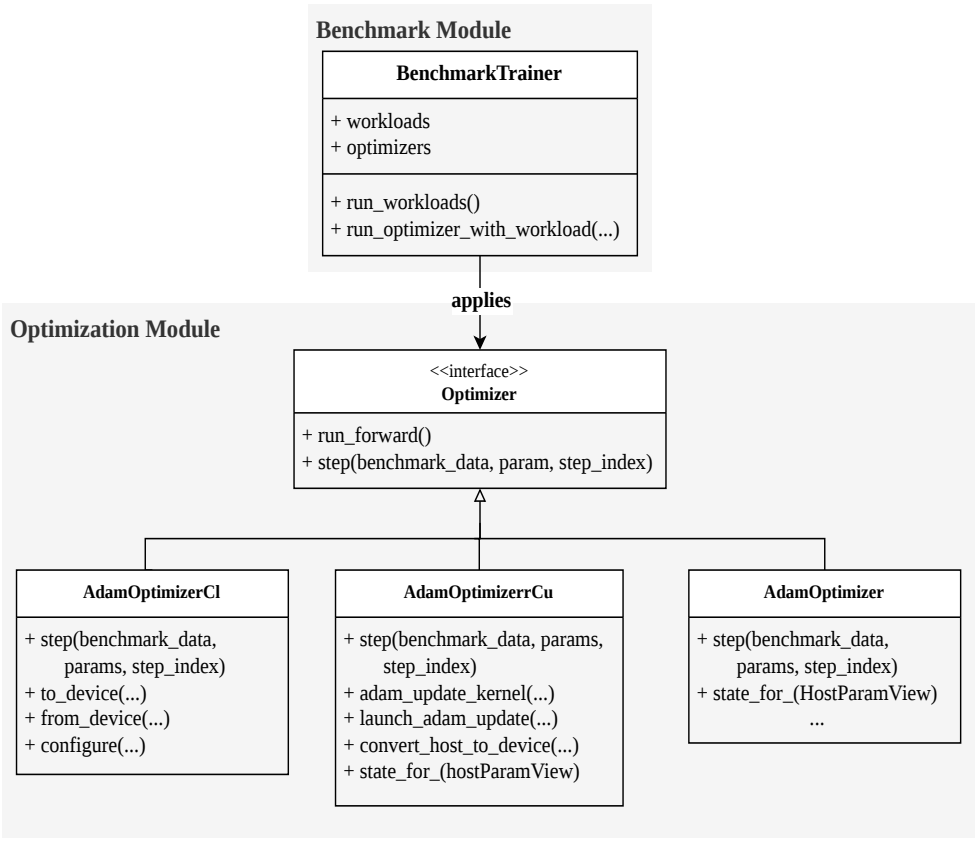

Figure 7: Domain-agnostic optimizer architecture supporting multiple implementations for different execution platforms.

Figure 7 illustrates the structure of the Optimization Layer. Each optimizer implements the abstract class `Optimizer` and must provide an implementation of the `step` function. The `step` function is executed during each iteration of the optimization process and updates parameters toward an optimum. For example, the Adam optimizer is implemented in the classes `AdamOptimizer`, `AdamOptimizerCl` (OpenCL), and `AdamOptimizerCu` (CUDA). The `step` function performs parameter updates at timestep t as described by Kingma and Ba [**adam**]. Because GPU data handling differs between frameworks, additional functions such as `fromDevice` and `launch_adam_kernel` are required.

4.1.3 Util Layer

The Util Layer provides shared functionality used throughout the application, including data views, logging, and framework wrappers.

Figure 8 shows the interactions between the utility components. Data view classes such as `HostParamsView` and `DeviceParamView` represent parameter data for CPU and GPU execution. Since OpenCL and CUDA require different device representations, parameter data must be converted from host to device format. This conversion is handled by the

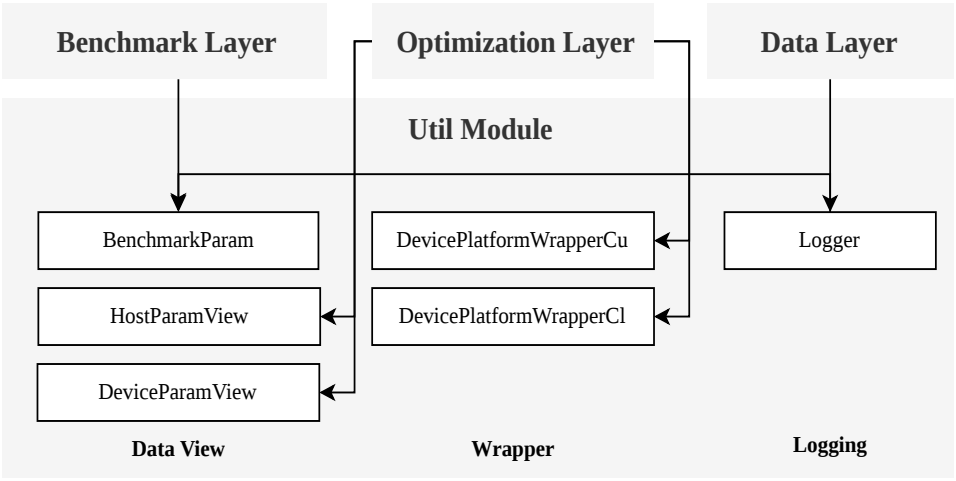

Figure 8: Utility components providing shared functionality across the application.

optimizer implementations. Framework wrapper classes encapsulate OpenCL and CUDA functionality, reducing the boilerplate code required for GPU execution. Benchmark results collected in the `benchmarkData` class are written to a CSV file using the `Logger` class.

4.1.4 Data Layer

The Data Layer stores benchmark results in a CSV file for later analysis. The logging system records both the execution context and performance measurements. Context information includes hyperparameter such as `learning_rate`, `beta1`, and `beta2`, while execution time is captured by the variable `time_ms`. Recording this contextual information enables more detailed analysis of relationships between configuration parameters and performance.

4.2 Measured Metrics

In this section, different metric categories and the corresponding key performance indicators are presented. The selected metrics follow the benchmarking best practices proposed by Beiranvand et al. [**best-practices-benchmark**].

4.2.1 Efficiency

One measured category is efficiency. The efficiency of an optimization algorithm describes the computational effort required to obtain a solution. Typical indicators include the number of function evaluations, execution time, and memory usage [**best-practices-benchmark**]. To improve reproducibility, execution time is measured in CPU time or GPU time. In contrast, wall-clock time can be influenced by hardware conditions and may therefore reduce reproducibility [**best-practices-benchmark**]. In addition to overall execution time, the benchmark measures both the GPU computation time of the workload and the data transfer time from the host (CPU) to the device (GPU). These measurements are performed for both CUDA and OpenCL in order to enable a direct comparison of the two

frameworks. In the current implementation, the optimizer update step is executed as a GPU kernel.

Figure **??** illustrates the execution flow of the benchmark program and highlights the parts executed on the GPU. Time measurements start when data is transferred to the GPU and stops whenever the transfer is done. The measurement for computation time of the kernel is measured during the execution of the optimizer update step. The update function is measured because it contains the core computations required to update the parameters toward the optimum in Adam.

4.2.2 Reliability

The reliability metric verifies that the implementations across the different frameworks behave as expected. In general, the reliability and robustness of an optimization algorithm describe its ability to consistently produce valid and stable results [**best-practices-benchmark**]. In this context, reliability is used to confirm the correctness of the implementation and to observe how execution time and data transfer time relate to the produced results. However, reliability is not treated as a key performance indicator in this report, since it primarily reflects the performance of the optimization algorithm itself rather than the performance of the underlying execution frameworks.

4.3 Hardware and Platforms

The benchmark was executed only on NVIDIA GPUs, namely an NVIDIA GeForce GTX 970 with 4 GB VRAM, an NVIDIA A100-SXM4 with 80 GB, and an NVIDIA Quadro RTX 5000 with 16 GB of VRAM. The A100-SXM4 is actually a GPU accelerator designed for High-Performance Computing (HPC) workloads. The RTX 5000 represents a more modern counterpart compared to the GeForce GTX 970. Since CUDA is only accessible on NVIDIA GPUs, the benchmark was executed exclusively on these devices. Consequently, this setup does not provide insights into the performance portability of OpenCL to GPUs from other vendors such as Intel or AMD.

Table 1 shows the hardware specifications of the GPUs used in the benchmark. All three devices originate from different time periods and represent milestones in GPU hardware development. It is also important to note that the GTX 970 is a consumer graphics card designed primarily for gaming and general graphics computation, whereas the A100 and the RTX 5000 are mainly used in high-performance computing environments, where the benchmark was also executed. The selection of these different hardware classes also motivates a separate presentation and interpretation of the benchmark results.

Table 1: Hardware specifications of the GPUs used in the benchmark. **[970]**[**5000**] [**A100**]

Spec	**GTX 970**	**A100-SXM4**	**RTX 5000**
General			
Year	2014	2020	2018
Architecture	Maxwell	Ampere	Turing
Launch price (USD)	329	8,000–10,000	2,299
Clock speeds			
Base Clock (MHz)	1050	1095	1620
Boost Clock (MHz)	1178	1410	1815
Memory			
Bandwidth (GB/s)	224.4	1560 (1.56 TB/s)	448.0
Memory Size (GB)	4	80	16
Render configuration			
Shading Units	1664	6912	3072
TMUs	104	432	192
ROPs	56	160	64
SM (M) Count	13	108	48
Tensor Cores	0	432	384
L1 Cache (KB)	48	192	64
L2 Cache (MB)	2	40	4

5 Benchmark results

5.1 Results for NVIDIA GeForce GTX 970

The benchmark program using the Adam optimizer and the **GEMM!** (**GEMM!**) workload was first executed on an NVIDIA GeForce GTX 970 with 8 GB of VRAM. For this benchmark, the following results were obtained for data transfer time and computation time using CUDA and OpenCL.

Table 2: Data transfer time and computation time across all batch indices

Metric	**CUDA**	**OpenCL**	**Perf. Ratio**
Average data-transfer time (ms)	4.06	4.43	? %
Average computation time (ms)	2.45	0.77	? %

Table 2 presents the benchmark results obtained with the specified hyperparameters. The data transfer time was averaged across all measurements performed for both OpenCL and CUDA. The results show that CUDA is 9% faster in terms of data transfer time on the NVIDIA GeForce GTX 970. Similarly, the computation time was averaged over all runs, resulting in OpenCL being 66% faster than CUDA.

Figure 9 shows the average computation time per `batch_index` and also includes the average loss per `batch_index` for both frameworks. This illustrates the efficiency of both frameworks while also providing an indication of their reliability. The figure shows that the loss decreases over time, which matches the expected behavior during optimization. Furthermore, the figure highlights the results summarized in Table 2. The data indicates that CUDA provides slightly better performance in terms of data transfer time, while

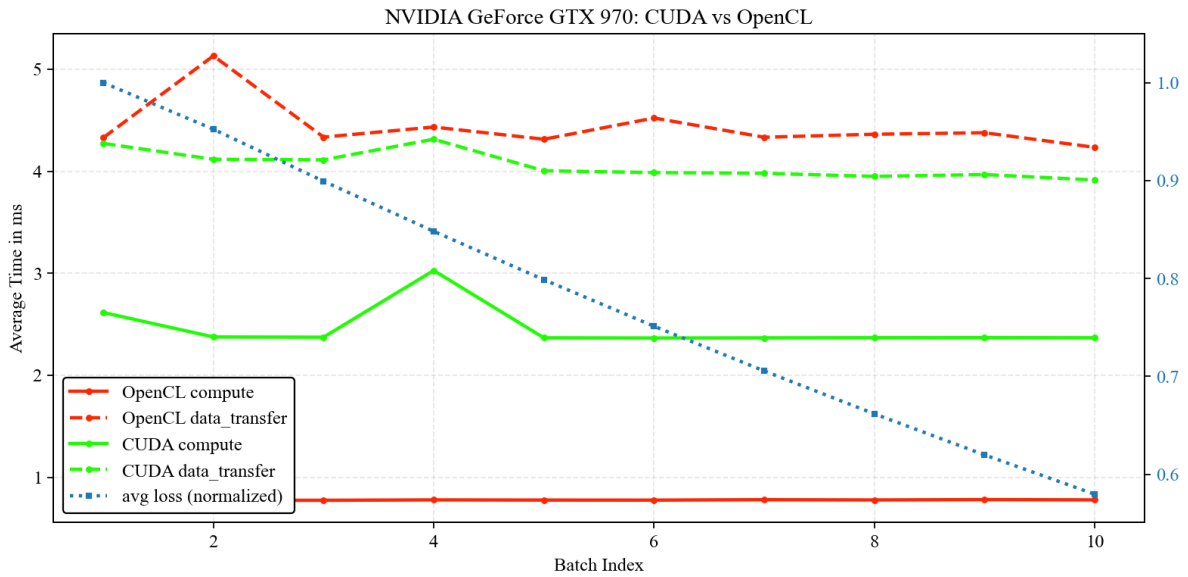

Figure 9: Results of running the GEMM workload using the Adam optimizer

OpenCL performs better in terms of computation time on the GTX 970, which represents the Maxwell architecture from NVIDIA.

5.2 Results for NVIDIA GeForce Quadro RTX 5000

The benchmark program was also executed on an NVIDIA Quadro RTX 5000, which is part of an HPC facility. Again, the Adam update step was measured in terms of data-transfer time and computation time for the GEMM workload.

Table 3: Data transfer time and computation time across all batch indices

Metric	**CUDA**	**OpenCL**	**Perf. Ratio**
Average data-transfer time (ms)	6.62	6.76	? %
Average computation time (ms)	0.30	0.28	? %

Table 3 shows the results of the workload executed on the RTX 5000. Again, the data-transfer time and computation time were averaged across all batch indices. On this platform, OpenCL performs slightly faster in terms of computation time, while CUDA performs slightly better in terms of data-transfer time.

Figure 10 further supports these results and validates the reliability of the benchmark. The decreasing loss curve, measured at each batch index, indicates that the optimization process behaves as expected.

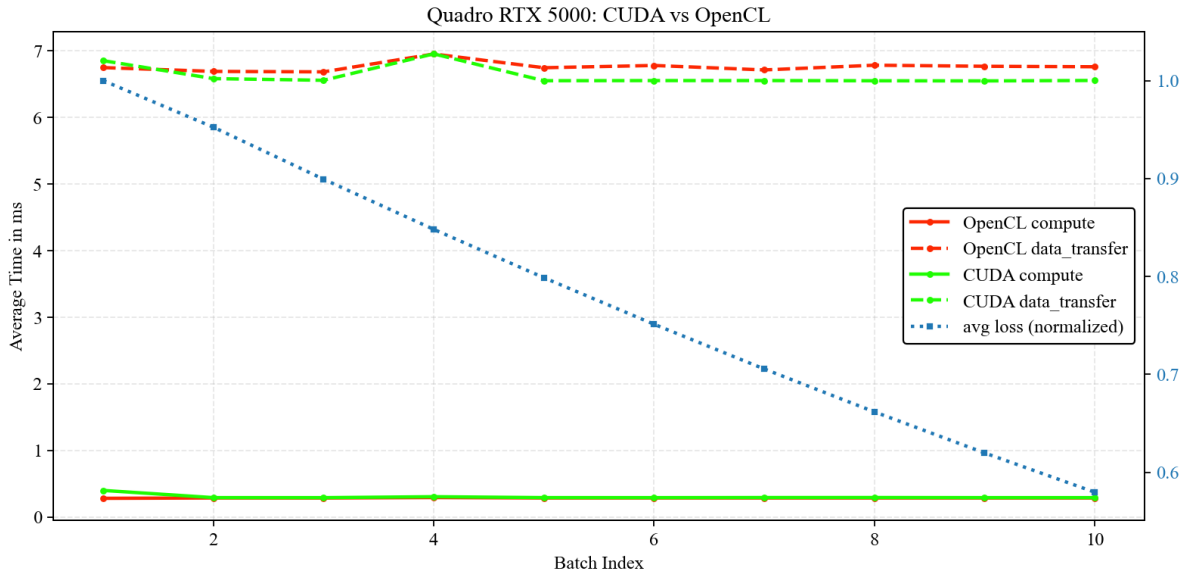

Figure 10: Results of running the GEMM workload using the Adam optimizer on NVIDIA Quadro RTX 5000

5.3 Results for NVIDIA A100-SXM4

The final benchmark was executed on the NVIDIA A100-SXM4, which is also primarily used in HPC environments. This GPU represents the most recent architecture included in the benchmark and is also the most powerful device according to the specifications shown in Table **??**.

Table 4: Data transfer time and computation time across all batch indices

Metric	**CUDA**	**OpenCL**	**Perf. Ratio**
Average data-transfer time (ms)	2.19	2.21	? %
Average computation time (ms)	0.11	0.08	? %

The results shown in Table 4 again indicate that OpenCL performs faster in terms of computation time, while CUDA performs slightly better in terms of data-transfer time.

Figure 11 again confirms these results and shows that, across the different batch index iterations, CUDA and OpenCL perform similarly on average on the A100. Furthermore, the average loss per batch index is presented to demonstrate the reliability of the optimization process during computation.

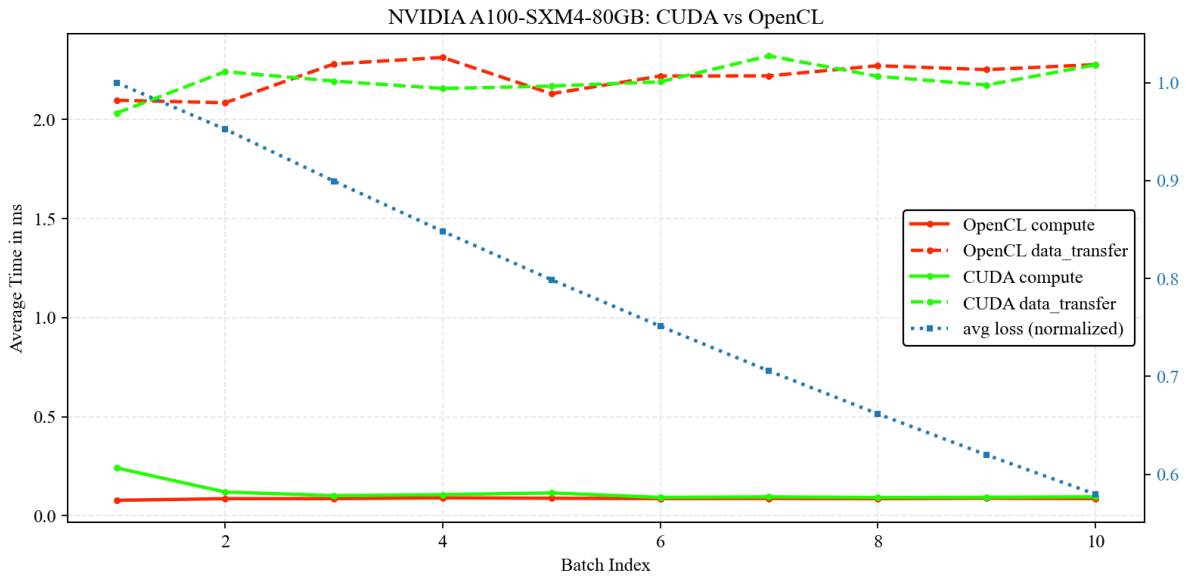

Figure 11: Results of running the GEMM workload using the Adam optimizer on NVIDIA A100

6 Discussion